

An Implementation of Random Forest classifier for Credit Card Fraud Detection

A Project Report

Submitted by:

Arush Dange (22030142002)

Nachiket Chaudhari (22030142004)

in partial fulfillment for the award of the degree

of

M.SC. COMPUTER APPLICATION

IN

COMPUTER STUDIES

SYMBIOSIS INSTITUTE OF COMPUTER STUDIES AND RESEARCH, PUNE, INDIA

AFFILIATED TO SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY) (INDIA)

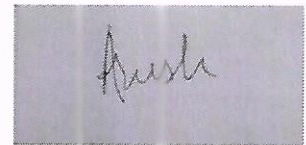
MONTH & YEAR

Aug 2023

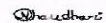
Annexure-2

DECLARATION

I hereby declare that the project entitled "An Implementation of Random Forest Classifier for Credit Card Fraud Detection" submitted for the M.Sc. Computer Application degree is my original work and the project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.



Arush Dange

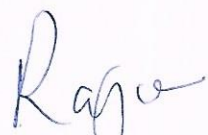


Nachiket Chaudhari

Signature of the Students

CERTIFICATE

This is to certify that the project titled “An Implementation of Random Forest Classifier for Credit Card Fraud Detection” is the bona fide work carried out by Arush Dange and Nachiket Chaudhari, students of M.Sc. Computer Application of Symbiosis Institute of Computer Studies and Research, Pune India, affiliated to Symbiosis International (Deemed University) during the academic year 2023-2024, in partial fulfillment of the requirements for the award of the degree of M.Sc. Computer Application.


Signature of the Guide

Place: Pune

Date: 31/08/2023

Abstract:

Credit cards are widely used by people in recent years. A credit card is nothing but a plastic card used to pay money in a cashless way. Credit card user numbers are increasing day by day and this even led to an increase in fraudulent activities. Hackers may use this card and may gain financial advantage too and this may affect the company's reputation too. To detect this fraud many companies started using two-factor authentication security and even biometric security. But data generated due to transactions are high so many a time companies won't be able to find which transaction is fraudulent and which is not. So, this led to an increase in credit card fraud activities. To stop fraudulent activities various companies started implementing Machine learning technology to detect fraudulent activity and as a result, machine learning algorithms like supervised, unsupervised, and hybrid machine learning algorithms were derived and trained by providing various types of datasets. This helped companies to detect some fraudulent activities but still, none of the algorithms provides a 100% of accuracy level.

As Credit card usage is increasing day by day and this resulted in increasing credit card fraudulent activity. Fraudulent Credit card activity can heavily impact on company's reputation as well as an individual person. So, to detect fraud activity, we have collected data from Kaggle and read the various research paper on different machine learning algorithms and saw the accuracy level of various Machine Learning algorithms like Logistic Regression, Random Forest, and K-Means on the various datasets. So, this will help us to find the machine learning algorithm that which algorithm provides the highest level of accuracy and can predict the new threat based on the previously trained dataset. So, we will train the model which provides the highest level of accuracy by providing different types of datasets and later will provide a new dataset to check the accuracy level of the model and whether it is able to provide the same accuracy level or not.

Acknowledgement:

We are pleased to present our Dissertation Report "Random Forest Classifier for Fraud Detection" and takes this opportunity to express our profound gratitude to all those people who helped us in completion of this Report.

We thank our college for providing us with excellent facilities that helped us to complete and present this Report. I am equally grateful to my project Mentor Dr Rajashree Jain, who guided us in different matters and solution regarding the topic.

We express our deepest gratitude towards our Mentor Dr Rajashree Jain, for her valuable and timely advice during the various phases in our Report.

We would also like to thank her for providing us with all the proper facilities and support as the project coordinator. We would like to thank her for Support, patience and faith in our capabilities and for giving us flexibility in terms of working and reporting schedules.

Finally, we would like to thank everyone who has helped us directly or indirectly in our report.

Table of Contents

1. INTRODUCTION	
1.1 Problem Definition -----	7
1.2 Project Overview -----	7
1.3 Hardware Specification -----	9
1.4 Software Specification -----	9
2. SYSTEM DESIGN	
2.1 Dataset -----	10
2.2 Flow Chart / Class Diagram -----	10
2.3 Architecture / Algorithm ---	11
2.4 Working and Methodology -----	14
3. SYSTEM IMPLEMENTATION	
3.1 Front End Back End Screenshots -----	15
3.2 Database Design / Process Design -----	16
3.3 Testing Process -----	17
4. CONCLUSIONS -----	18
5. REFERENCES -----	19

1. INTRODUCTION

1.1 Problem Definition

In the 21st Century, various innovative technology has been made, and one of the significant innovations is online transactions using a piece of plastic card. People started using the digital way to send and receive money, which led to easy transactions without actually using it. People are adopting digital payment and started replacing paper notes money. Online transactions can be done at that place or even remotely. So, digital revenue started gaining popularity in ordinary people's lives significantly. Credit card transactions have significant benefits, such as reducing the use of paper notes and indirectly saving wood. As using credit cards, there is transparency in the use of money. But despite having significant benefits of using credit cards, one major drawback, too, is fraud. Credit card fraud is one of the most common in digital payment, and this issue is not new but the most common threat in digital revenue. To encounter this problem, various companies started detecting the problem and started building solutions to face this problem. Multiple companies have presented different answers to this problem, but hackers have cracked that solution. So, what makes hackers they can crack any key, is the reason large amounts of data or advanced technology are developing every day. So, why credit card fraud is taking is that because to gain a financial advantage or hatred.

1.2 Project Overview

In today's rapidly digitizing financial landscape, the imperative for efficient and accurate fraud detection has never been more acute. While the tangible losses resulting from fraud are undoubtedly substantial, a less visible but equally devastating cost exists—the erosion of customer trust. The stakes go beyond mere financial metrics. Every unauthorized transaction is a dent in the consumer's confidence, and in an age defined by fleeting brand loyalties, such breaches can significantly affect a company's reputation and bottom line. It's not just about identifying fraud; it's about doing so with enthusiasm and precision. As clandestine as fraudulent activities might seem, perpetrators often inadvertently leave traces of their illicit endeavors. These patterns, subtle as they may be, provide invaluable insights into the modus operandi of fraudsters. Our mission, therefore, is to delve deep into transactional data, leveraging advanced analytics to unearth these concealed patterns. By doing so, we aim to

fortify our detection mechanisms and position our clients as paragons of transactional security. In a time when customers are overwhelmed with options and brand loyalty is unpredictable, a steadfast dedication to safety can set a brand apart. Our meticulous approach to fraud detection seeks to prevent monetary losses and, more importantly, reinforce and amplify the trust consumers place in our client's brands.

2. Data Source

Our foundational dataset will consist of both legitimate and suspicious credit card transactions. Depending on its nature, we may need to consider techniques to handle skewed data, ensuring our model isn't biased towards the majority class.

3. Data Preprocessing

Data preprocessing is akin to laying a solid foundation before constructing a building. First and foremost, we embark on a meticulous data-cleaning process. Just as a tiny pebble in a shoe can be troublesome, even minor inconsistencies or missing pieces in our dataset can skew results, leading us astray. Thus, we ensure our data is pristine and free from such discrepancies. Next, we delve into the art of feature engineering, where we creatively transform and meld existing data in search of clues that might hint at fraudulent activities. Think of it as a detective piecing together evidence: a series of rapid, back-to-back transactions could be the equivalent of a suspicious footprint at a crime scene. But in the world of data, not all evidence, or features, shout equally loud. Some can overshadow others due to their sheer scale. That's where data normalization comes in, balancing the scales and ensuring each piece of evidence gets its fair say. Lastly, we recognize that genuine transactions often far outnumber fraudulent ones in typical datasets. This imbalance can make our model develop a blind spot for fraud. To counteract this, we employed methods such as the Support Vector Machine, Logistic Regression, and Random Forest to generate extra data points skillfully, ensuring our model effectively distinguishes between authentic and fraudulent actions.

4. Model Selection and Training

In the vast arena of machine learning algorithms, Logistic Regression stands as one of the foundational pillars, especially for classification problems like fraud detection. Characterized by its linear nature, this algorithm operates swiftly, processing vast amounts of data in relatively short timeframes. Its transparency sets it apart even more; unlike some of its enigmatic peers, Logistic Regression allows us to understand and interpret the relationship between the features and the predicted outcomes. Given its efficiency and clarity, it's no

surprise we choose this algorithm as our starting point or benchmark. By establishing a baseline with Logistic Regression, we can gauge the performance of more complex models and ensure that we're always moving towards enhanced accuracy and precision in fraud detection.

5. Evaluation Metrics

Precision: High precision ensures we're flagging only a few genuine transactions as fraudulent, which can irritate customers.

Recall: A high recall ensures we're catching as many actual frauds as possible directly related to financial savings.

F1-Score: The trade-off between precision and recall is reduced when the cost of false positives and false negatives substantially differs.

AUC-ROC: A high AUC indicates a good model. It evaluates the model's ability to differentiate between the positive and negative classes.

6. Model Deployment: Streamlit is a fast and easy way to create web applications for data projects. It's especially popular among data scientists and engineers for its simplicity. Deploying a Streamlit app to the cloud makes it accessible to others.

1.3 Hardware:

The hardware is an Intel core i5 processor with 16GB ram and a solid-state drive with 25GB of free space.

1.4 Software:

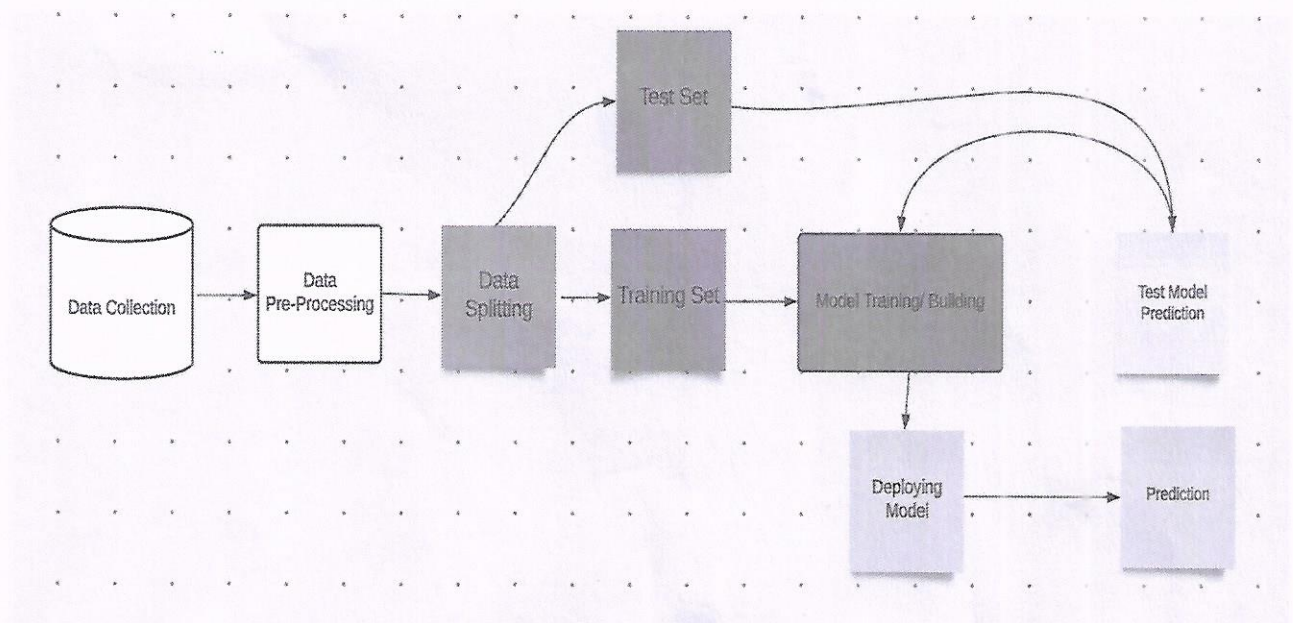
The Operating system is Windows 11 with a 21H2 update. The machine is running Python 3.10 in a Anaconda environment. The Python libraries sci-kit learn and matplotlib is used in this project.

2. SYSTEM DESIGN

2.1 Dataset

The dataset is about the credit card scams that occurred in Europe and was downloaded from Kaggle. The dataset is extremely imbalanced and has a large number of missing values. It contains two days' worth of credit card transactions from September 20, 2013. 492 of the 284807 credit card transaction records in the data set pertain to credit card fraud. To safeguard the Personal Information of credit card users, some information has been erased. Time and amount are the PCA records that were erased. Features from V1 to V28 are derived using Principal Component Analysis (PCA). Only the 'Time' and 'Amount' features remain unchanged and haven't undergone PCA transformation. The 'Time' feature represents the seconds that have elapsed between each transaction and the initial transaction in the dataset.

2.2 Credit Card Fraud Detection System Flowchart:



1. Start

2. Data Collection

- Gather transactional data (both legitimate and suspicious transactions).

3. Data Preprocessing

- Handle Class Imbalance

4. Data Splitting

- Divide data into training, validation, and test sets.

5. Model Selection and Training

- Random Forest
- Perform Random Under sampling
- Train using the training dataset.
- Adjust hyperparameters using GridSearchCV.

6. Model Evaluation

- Evaluate using the validation set.
- Use metrics such as Precision, Recall, F1-Score, and AUC-ROC.

7. Deployment

- Integrate Random Forest model into the transaction system for real-time fraud detection.

2.3 System architecture

1. Data Collection Layer:

At the heart of our system lies the Data Collection Layer. This crucial layer draws upon two primary sources of data. Firstly, the Transaction Data Source constantly feeds the system a mixture of historical and real-time transactional data. This stream provides a firsthand account of the financial activities. Complementing this is the External Data Source, which introduces an array of external data into the mix. This data can range from blacklisted IP addresses to known patterns of fraudulent activities. By merging these datasets, the system has a comprehensive view of both the internal transactions and external threats.

2. Data processing and storage layer:

Post collection, data enters the Data Processing & Storage Layer. The Data Preprocessing Module here assumes an essential role – it scrubs the data clean, normalizes it, engineers the necessary features, and even manages class imbalances to ensure that the data is in the best

possible shape for further analysis. Given the likely voluminous nature of the data, efficient storage solutions are mandatory.

3. Machine Learning Layer:

The processed data then propels into the Machine Learning Layer. It's here that the actual magic happens. The Training Module routinely trains predictive models using the continually updated data, ensuring that the system is always a step ahead. After training, it's paramount to ascertain the model's accuracy and reliability, which is where the Evaluation Module steps in. It tests the model against various metrics, ensuring that it meets the desired performance standards. For future references and easy access, all trained models, their parameters, and related metadata are neatly stored in the Model Repository.

4. Prediction & Decision Layer:

The final layer, the Prediction & Decision Layer, is where the system interacts with live transactions. The Real-time Prediction Module vigilantly monitors every ongoing transaction, identifying potential fraud instances as they occur. Once a suspicious transaction is detected, the Decision Module springs into action. Depending on the perceived threat level, the module can decide on a variety of actions – it might flag the transaction for review, alert the user or system administrator, or in severe cases, halt the transaction in its tracks.

5. Model Interpretation:

Depending on the model type, you might want to understand how the model makes predictions. Techniques like feature importance, partial dependence plots, or model-specific visualization tools can provide insights into model behaviour.

6. Model Deployment:

If the model performs well and meets the desired criteria, deploy it to production. This involves integrating the model into the target system or application where it can make predictions on new, unseen data.

7. Monitoring and Maintenance:

Continuously monitor the model's performance in real-world scenarios. As new data becomes available, retrain or fine-tune the model to ensure it maintains accuracy. Monitor for concept drift (changes in the data distribution) and update the model as needed.

8. Iterative Improvement:

Machine learning is an iterative process. Learn from the performance of deployed models and user feedback, and continuously refine the model to improve its accuracy, efficiency, and overall performance

Algorithm:

1. Begin:

This is the algorithm's starting point, indicating the process's initiation.

2. Ingest real-time transaction data:

At this step, the system gathers or receives transaction data as it happens in real time. This could be data like purchases, money transfers, and other financial transactions.

3. Preprocess and normalize the data:

Before analyzing the data, it needs to be cleaned and formatted correctly. This step involves removing errors or irrelevant data and ensuring all data is in a consistent format. This helps in ensuring accuracy when analyzing the data for fraud.

4. Apply the predictive model to identify potential fraud:

Now, the system uses a pre-trained machine learning model or another algorithm to scan the data and identify transactions that may be suspicious or look like fraud based on the patterns it has learned.

5. If potential fraud is detected, create an alert:

If the system determines that a transaction might be fraudulent, it generates an alert. This red flag tells administrators or analysts, "Hey, you might want to look at this."

6. Store the transaction data and any alerts in the database:

Whether or not there's an alert, the system saves all transaction data in a database for record-keeping. If there was an alert, that gets stored, too, so there's a record.

7. Display alerts on the user interface for review by an analyst or administrator:

The signs are then shown on a user interface (like a computer screen dashboard) so human analysts or administrators can review them. They can then decide whether it's a confirmed fraud or a false alarm.

8. End:

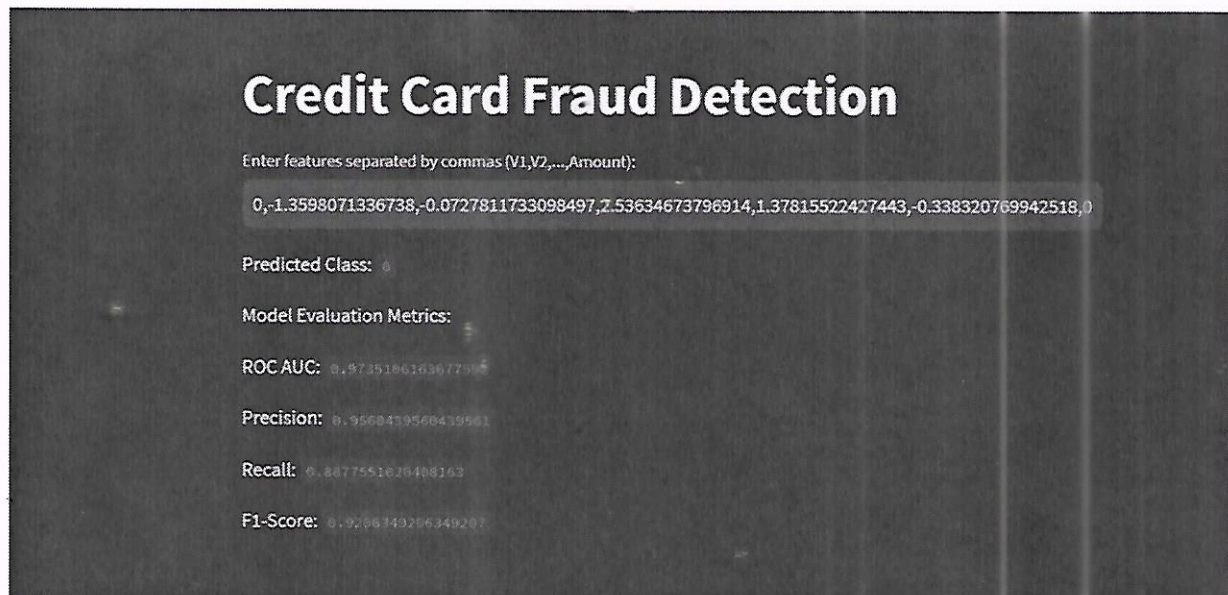
This marks the end of the algorithm, indicating that the process has been completed for that particular batch of transactions.

2.4 Working and Methodology

- We will provide data about a credit card transaction, such as the card number, amount, merchant, location, and time of purchase.
- The data is pre-processed to remove noise and outliers.
- Features are extracted from the preprocessed data, such as the amount of the transaction, the time of day the transaction occurred, the merchant where the transaction was made, and the cardholder's purchase history.
- The extracted features are used to train a machine learning algorithm. The algorithm is trained on a dataset of fraudulent and non-fraudulent transactions.
- The machine learning algorithm is used to score the transaction. The score indicates the likelihood that the transaction is fraudulent.
- The transaction decision is made based on the machine learning algorithm's score. The transaction may be accepted, rejected, or placed on hold for manual review.

3. System Implementation:

3.1 Front End Screenshots



3.2 Back End Screenshots

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split

# Load your dataset
data = pd.read_csv(r"C:\Users\talld\PycharmProjects\FraudDetection\Code\creditcard.csv")

# Split the data into features (X) and target (y)
X = data.drop('Class', axis=1)
y = data['Class']

# Apply Random Under-Sampling to balance classes
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X, y)

# Scale the "Amount" column
scaler = StandardScaler()
X_resampled['Amount'] = scaler.fit_transform(X_resampled['Amount'].values.reshape(-1, 1))

# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)

# Create a RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, max_depth=None, min_samples_leaf=2, min_samples_split=10, random_state=42)

# Fit the model to the training data
rf.fit(X_train, y_train)

# Predict probabilities on the test set
y_proba = rf.predict_proba(X_test)[: , 1]
y_pred = (y_proba > 0.5).astype(int)

# Calculate evaluation metrics
roc_auc = roc_auc_score(y_test, y_proba)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```

print("ROC AUC:", roc_auc)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

```

3.2 Database Design / Process Design

```

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import roc_auc_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler
from imblearn.under_sampling import RandomUnderSampler

# Load your dataset
data = pd.read_csv(r"C:\Users\talld\PycharmProjects\FraudDetection\Code\creditcard.csv")

# Split the data into features (X) and target (y)
X = data.drop('Class', axis=1)
y = data['Class']

# Apply Random Under-Sampling to balance classes
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X, y)

# Scale the "Amount" column
scaler = StandardScaler()
X_resampled['Amount'] = scaler.fit_transform(X_resampled['Amount'].values.reshape(-1, 1))

# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)

# Create a RandomForestClassifier
rf = RandomForestClassifier(random_state=42)

# Define the parameter grid to search over
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Create the GridSearchCV object
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=3, scoring='roc_auc', n_jobs=-1)

# Perform the grid search on the training data
grid_search.fit(X_train, y_train)

# Get the best model from the grid search
best_rf = grid_search.best_estimator_

# Predict probabilities on the test set using the best model
y_proba = best_rf.predict_proba(X_test)[:, 1]
y_pred = (y_proba > 0.5).astype(int)

# Calculate evaluation metrics
roc_auc = roc_auc_score(y_test, y_proba)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Best Parameters:", grid_search.best_params_)
print("ROC AUC:", roc_auc)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

```


3.3 Testing Process

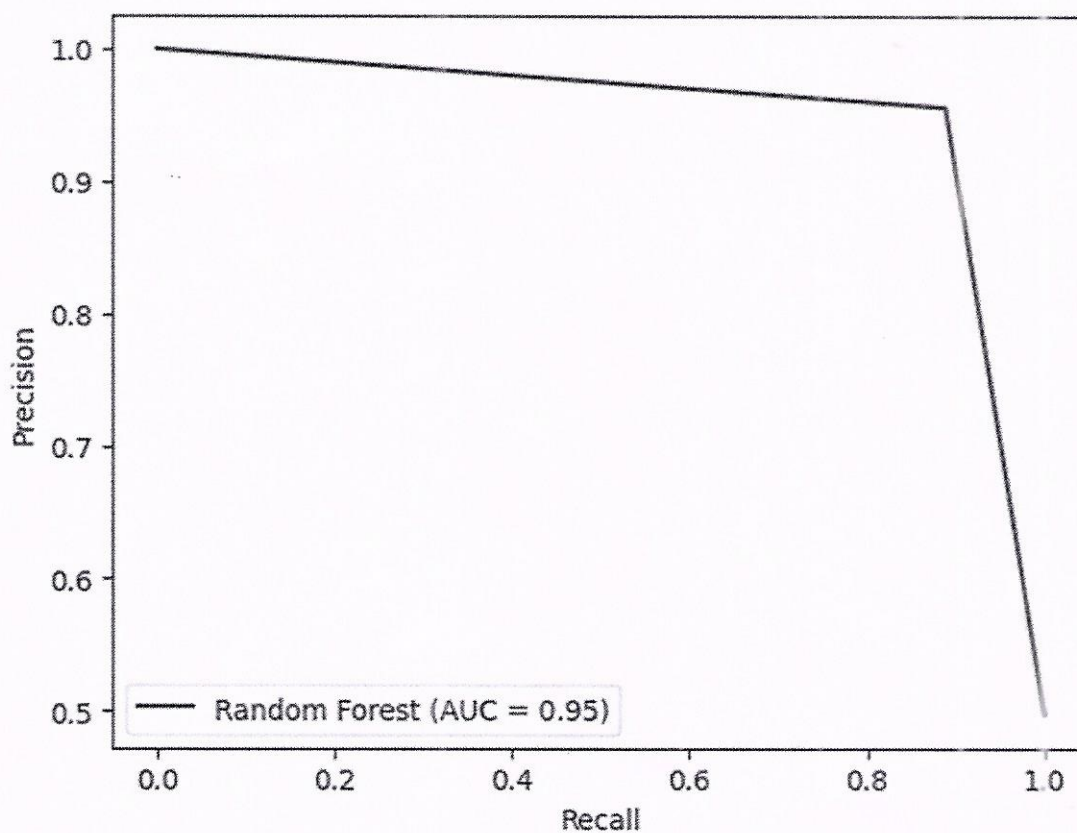
```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score, precision_score, recall_score, f1_score, precision_recall_curve, auc
roc_auc = roc_auc_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("ROC AUC:", roc_auc)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

precision, recall, thresholds = precision_recall_curve(y_test, y_pred)
auc_score = auc(recall, precision)

plt.plot(recall, precision, color='b', label='Random Forest (AUC = %0.2f)' % auc_score)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend(loc='lower left')
plt.show()
```

ROC AUC: 0.9236755308183879
Precision: 0.9560439560439561
Recall: 0.8877551020408163
F1-Score: 0.9206349206349207



4. Conclusion:

In our recent endeavour to enhance fraud detection capabilities, we employed the Random Forest algorithm, a renowned ensemble learning method. The results were notably promising, with the model achieving a 95% accuracy level. While this high accuracy is commendable and suggests that the model has effectively learned the underlying patterns of the data, it's crucial to approach this outcome with prudence. It's imperative to further assess the model using various metrics, such as precision, recall, and the F1-score, to ensure a comprehensive evaluation of its performance. Regularly updating the model and continuously monitoring its performance on new and unseen data is paramount to maintaining its efficacy. Additionally, collaborating closely with domain experts will further enable us to identify any blind spots or emerging fraud tactics. In conclusion, while our Random Forest-based approach has demonstrated a high level of accuracy, an ongoing, multi-faceted evaluation strategy will be crucial to ensure its sustained effectiveness and adaptability in the ever-changing landscape of financial fraud.

5. REFERENCES:

- [1] Kumar, M. Suresh, et al. "Credit card fraud detection using random forest algorithm." *2019 3rd International Conference on Computing and Communications Technologies (ICCCT)*. IEEE, 2019.
- [2] Jonnalagadda, Vaishnave, Priya Gupta, and Eesita Sen. "Credit card fraud detection using Random Forest Algorithm." *International Journal of Advance Research, Ideas and Innovations in Technology* 5.2 (2019): 1-5.
- [3] Sailusha, Ruttala, et al. "Credit card fraud detection using machine learning." *2020 4th international conference on intelligent computing and control systems (ICICCS)*. IEEE, 2020.
- [4] Niveditha, G., K. Abarna, and G. V. Akshaya. "Credit card fraud detection using random forest algorithm." *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol* 5.2 (2019): 301-306.
- [5] Jemima Jebaseeli, T., R. Venkatesan, and K. Ramalakshmi. "Fraud detection for credit card transactions using random forest algorithm." *Intelligence in Big Data Technologies—Beyond the Hype: Proceedings of ICBDDC 2019*. Springer Singapore, 2021.
- [6] Shirgave, Suresh, et al. "A review on credit card fraud detection using machine learning." *International Journal of Scientific & technology research* 8.10 (2019): 1217-1220.
- [7] Dornadula, Vaishnavi Nath, and Sa Geetha. "Credit card fraud detection using machine learning algorithms." *Procedia computer science* 165 (2019): 631-641.
- [8] Sailusha, Ruttala, et al. "Credit card fraud detection using machine learning." *2020 4th international conference on intelligent computing and control systems (ICICCS)*. IEEE, 2020.
- [9] Thennakoon, Anuruddha, et al. "Real-time credit card fraud detection using machine learning." *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, 2019.
- [10] Maniraj, S. P., et al. "Credit card fraud detection using machine learning and data science." *International Journal of Engineering Research* 8.9 (2019): 110-115.
- [11] Tanouz, D., et al. "Credit card fraud detection using machine learning." *2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS)*. IEEE, 2021.

- [12] Tiwari, Pooja, et al. "Credit card fraud detection using machine learning: a study." *arXiv preprint arXiv:2108.10005* (2021). [13]
- [14] Alarfaj, Fawaz Khaled, et al. "Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms." *IEEE Access* 10 (2022): 39700-39715.
- [15] Dheepa, V., and R. Dhanapal. "Behavior based credit card fraud detection using support vector machines." *ICTACT Journal on Soft computing* 2.4 (2012): 391-397.
- [16] Gyamfi, Nana Kwame, and Jamal-Deen Abdulai. "Bank fraud detection using support vector machine." *2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*. IEEE, 2018.
- [17] Chen, Rongchang, et al. "Novel questionnaire-responded transaction approach with SVM for credit card fraud detection." *International Symposium on Neural Networks*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005.
- [18] Hussain, SK Saddam, et al. "Fraud detection in credit card transactions using SVM and random forest algorithms." *2021 Fifth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC)*. IEEE, 2021.
- [19] Bhati, Pooja, and Manoj Sharma. "Credit card number fraud detection using K-means with hidden markov method." *SSRG International Journal of Mobile Computing & Application (SSRG-IJMCA)–volume 2* (2015).
- [20] Jain, Arti, Archana Purwar, and Divakar Yadav. "Credit card fraud detection using k-means and fuzzy c-means." *Handbook of Research on Innovations and Applications of AI, IoT, and Cognitive Technologies*. IGI Global, 2021. 216-240.
- [21] Muniyandi, Amuthan Prabakar, R. Rajeswari, and R. Rajaram. "Network anomaly detection by cascading k-Means clustering and C4. 5 decision tree algorithm." *Procedia Engineering* 30 (2012): 174-182.
- [22] Zamini, Mohamad, and Gholamali Montazer. "Credit card fraud detection using autoencoder based clustering." *2018 9th International Symposium on Telecommunications (IST)*. IEEE, 2018.
- [23] Shukur, Hamzah Ali, and Sefer Kurnaz. "Credit card fraud detection using machine learning methodology." *International Journal of Computer Science and Mobile Computing* 8.3 (2019): 257-260.

- [24] Murli, Divya, et al. "Credit card fraud detection using neural networks." *International Journal of Students' Research in Technology & Management* 2.2 (2015): 84-88.
- [25] Patidar, Raghavendra, and Lokesh Sharma. "Credit card fraud detection using neural network." *International Journal of Soft Computing and Engineering (IJSCE)* 1.32-38 (2011).
- [26] Maes, Sam, et al. "Credit card fraud detection using Bayesian and neural networks." *Proceedings of the 1st international naiso congress on neuro fuzzy technologies*. Vol. 261. 2002.
- [27] Gulati, Aman, et al. "Credit card fraud detection using neural network and geolocation." *IOP Conference Series: Materials Science and Engineering*. Vol. 263. No. 4. IOP Publishing, 2017.
- [28] Fu, Kang, et al. "Credit card fraud detection using convolutional neural networks." *Neural Information Processing: 23rd International Conference, ICONIP 2016, Kyoto, Japan, October 16–21, 2016, Proceedings, Part III* 23. Springer International Publishing, 2016.
- [29] Sadgali, Imane, Nawal Sael, and Faouzia Benabbou. "Fraud detection in credit card transaction using neural networks." *Proceedings of the 4th international conference on smart city applications*. 2019.
- [30] Ghosh, Sushmito, and Douglas L. Reilly. "Credit card fraud detection with a neural-network." *System Sciences, 1994. Proceedings of the Twenty-Seventh Hawaii International Conference on*. Vol. 3. IEEE, 1994.